

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

```
import math
import random
import tkinter as tk
from tkinter import messagebox

# ----- Helper Cryptographic Functions -----
print("Hariprasad Vishwakarma")

def generate_prime():
    """Generates a small prime number for the GUI demo."""
    primes = [61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151]
    return random.choice(primes)

def egcd(a, b):
    """Corrected Extended Euclidean Algorithm."""
    if a == 0:
        return b, 0, 1
    g, x1, y1 = egcd(b % a, a)
    # The fix: update the coefficients correctly based on recursion tracking
    x = y1 - (b // a) * x1
    y = x1
    return g, x, y

def mod_inverse(e, phi):
    """Calculates modular inverse using the corrected egcd."""
    g, x, y = egcd(e, phi)
    if g != 1:
        return None
    return x % phi

# ----- Core RSA & UI Interaction Logic -----

# A simple dictionary to store key states securely without relying on broad 'global' keywords
keys = {"e": None, "d": None, "n": None}

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()
    Hariprasad Vishwakarma
    T127
    TYCS
```

```

n_val = p * q
phi = (p - 1) * (q - 1)

e_val = 3
while math.gcd(e_val, phi) != 1:
    e_val += 2

d_val = mod_inverse(e_val, phi)

# Save values to our state manager
keys["e"] = e_val
keys["d"] = d_val
keys["n"] = n_val

public_key_var.set(f"({e_val}, {n_val})")
private_key_var.set(f"({d_val}, {n_val})")

# Reset input fields on new key generation
cipher_var.set("")
decrypted_var.set("")

def encrypt():
    if keys["n"] is None:
        messagebox.showerror("Error", "Please generate keys first.")
        return

    try:
        msg = int(message_entry.get())

        if msg >= keys["n"]:
            messagebox.showerror("Error", f"Message must be smaller than n ({keys['n']}).")
            return

        cipher = pow(msg, keys["e"], keys["n"])
        cipher_var.set(str(cipher))

    except ValueError:
        messagebox.showerror("Error", "Enter a valid integer message.")

def decrypt():
    if keys["d"] is None:
        messagebox.showerror("Error", "No private key available for decryption.")
        return

```

Hariprasad Vishwakarma

T127

TYCS

```

try:
    cipher = int(cipher_var.get())
    plain = pow(cipher, keys["d"], keys["n"])
    decrypted_var.set(str(plain))

except ValueError:
    messagebox.showerror("Error", "Invalid Cipher Text. Enter a valid integer.")

```

----- GUI Architecture -----

```

root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("500x420")
root.resizable(False, False)

public_key_var = tk.StringVar()
private_key_var = tk.StringVar()
cipher_var = tk.StringVar()
decrypted_var = tk.StringVar()

tk.Label(root, text="RSA Algorithm Playground", font=("Arial", 16, "bold")).pack(pady=10)

tk.Button(root, text="Generate Keys", command=generate_keys, bg="lightblue",
padx=10).pack(pady=5)

tk.Label(root, text="Public Key (e, n)", font=("Arial", 9, "bold")).pack()
tk.Entry(root, textvariable=public_key_var, width=50, state="readonly",
justify="center").pack(pady=2)

tk.Label(root, text="Private Key (d, n)", font=("Arial", 9, "bold")).pack()
tk.Entry(root, textvariable=private_key_var, width=50, state="readonly",
justify="center").pack(pady=2)

tk.Label(root, text="Enter Numeric Message", font=("Arial", 9, "bold")).pack(pady=5)
message_entry = tk.Entry(root, width=30, justify="center")
message_entry.pack()

tk.Button(root, text="🔒 Encrypt Message", command=encrypt, bg="lightgreen",
padx=10).pack(pady=5)

tk.Label(root, text="Cipher Text Output", font=("Arial", 9, "bold")).pack()
tk.Entry(root, textvariable=cipher_var, width=40, justify="center").pack(pady=2)

```

Hariprasad Vishwakarma

T127

TYCS

```
tk.Button(root, text="🔓 Decrypt Cipher", command=decrypt, bg="orange",
padx=10).pack(pady=5)
```

```
tk.Label(root, text="Decrypted Message Result", font=("Arial", 9, "bold")).pack()
tk.Entry(root, textvariable=decrypted_var, width=40, state="readonly",
justify="center").pack(pady=2)
```

```
root.mainloop()
```

Output:

